❓ Esercizi Modulo 2

✏️ PyTorch, Automatic Differentiation, and Training Mechanics

Tempo previsto: *circa 30 minuti.*

L'obiettivo di questo esercizio è comprendere, attraverso un piccolo esperimento in **PyTorch**, perché il **campo ricettivo** (*receptive field*) di una rete convoluzionale sia importante nell'imaging computazionale. Questa idea diventerà fondamentale più avanti, quando verranno introdotte architetture più profonde e modelli di tipo **UNet**.

Considera lo scenario seguente. Parti da un'immagine pulita in scala di grigi $\boldsymbol{x}$, crea una versione sfuocata $\boldsymbol{y} = K\boldsymbol{x}$, e addestra una piccola CNN per predire $\boldsymbol{x}$ a partire da $\boldsymbol{y}$. Confronta due reti:

- **Modello A:** una CNN poco profonda (*shallow*) con pochi livelli `Conv2d` e senza pooling;
- **Modello B:** una CNN che include almeno un passaggio di downsampling (per esempio `MaxPool2d` o una convoluzione con stride 2) e un passaggio di upsampling.

Lo scopo non è ottenere la migliore qualità di ricostruzione, ma osservare come l'ampliamento del campo ricettivo effettivo modifichi il risultato.

▼ Task 1: Build the dataset

Utilizza le idee già mostrate in questo notebook.

1. Inizia da una o più immagini pulite disponibili nel materiale del corso.
2. Costruisci un piccolo sottoinsieme di addestramento con un dataloader, per esempio selezionando un numero limitato di immagini Mayo.
3. Corrompile con un operatore di sfuocamento (*blur*). Puoi usare l'operatore di blurring di `IPPy` o un `Conv2d` standard con kernel fisso.
4. Tieni da parte un piccolo set di validazione.

▼ Task 2: Define two reconstruction models

Implementa due modelli in **PyTorch**.

1. Il **Modello A** dovrebbe utilizzare solo convoluzioni locali e nonlinearità puntuali.
2. Il **Modello B** dovrebbe includere un meccanismo che amplia il campo ricettivo (*receptive field*), come il pooling o la convoluzione con stride, seguito da un passaggio di upsampling.
3. Mantieni entrambi i modelli sufficientemente piccoli affinché l'addestramento sia rapido.

▼ Task 3: Train both models

Addestra le due reti sugli stessi dati.

1. Usa la stessa funzione di perdita (*loss function*) per entrambi i modelli, ad esempio l'MSE (Mean Squared Error).
2. Usa lo stesso ottimizzatore e all'incirca lo stesso numero di iterazioni di addestramento.
3. Registra la loss di addestramento e, se possibile, anche la loss di validazione.

▼ Task 4: Compare the outputs

Scegli uno o due esempi di validazione e visualizza:

1. l'immagine pulita,
2. l'input sfuocato (*blurred*),
3. la ricostruzione prodotta dal **Modello A**,
4. la ricostruzione prodotta dal **Modello B**.

Poi rispondi alle seguenti domande in un breve paragrafo:

1. Quale modello ricostruisce in modo più efficace le strutture sfuocate di grandi dimensioni?
2. Quale ruolo sembra giocare il campo ricettivo?
3. Qual è il possibile svantaggio del pooling in un problema di imaging dove i dettagli spaziali fini sono importanti?

▼ Deliverable

Consegna i seguenti quattro elementi:

1. il codice che definisce i due modelli;
2. una figura che mostri le curve della loss di addestramento (*training-loss*);
3. una figura che confronti le ricostruzioni su almeno un esempio di validazione;
4. una breve discussione scritta di circa 8-10 righe che spieghi quanto osservato.

Un elaborato completo dovrebbe permettere di verificare sia che i modelli siano stati implementati correttamente, sia che tu abbia compreso la connessione tra **campo ricettivo** (*receptive field*), **pooling** e ricostruzione dell'immagine.

✏️ Convolutions, CNNs, and End-to-End Training

▼ **Esercizio di codice:** modifica il numero di filtri in `SimpleCNN` e confronta sia il numero totale di parametri che la qualità della ricostruzione.

Il codice originale definisce `SimpleCNN` e calcola 9857 parametri totali impostando `n_filters = 32`.

- **Aumento del numero di filtri (es. `n_filters = 64`):** Aumentando i filtri in `self.conv1`, `self.conv2`, e `self.conv3`, la capacità espressiva della rete cresce, poiché la rete può estrarre più "pattern" in parallelo (più caratteristiche e combinazioni visive). Il numero totale di parametri (pesi del kernel e bias) aumenterà quadraticamente nei livelli intermedi. La qualità della ricostruzione dovrebbe migliorare (la rete sarà in grado di rimuovere meglio il blur), ma il modello sarà più lento da addestrare e potrebbe essere più soggetto a overfitting se i dati non sono sufficienti.
- **Riduzione del numero di filtri (es. `n_filters = 16` o `8`):** Il numero di parametri diminuirà drasticamente (meno di 9857). La rete sarà più veloce da addestrare, ma la qualità della ricostruzione subirà un grave peggioramento. Come indicato nel documento, le scarse prestazioni visive della `SimpleCNN` base sono dovute proprio al fatto che "il modello è troppo piccolo e ha troppi pochi parametri" per invertire efficacemente l'operatore K.

▼ **Esercizio di codice:** modifica la dimensione del kernel (*kernel size*) dei livelli convoluzionali e osserva come cambia il comportamento del modello.

Il modello originale usa `kernel_size = 3`, ovvero filtri 3×3.

- **Diminuzione a `kernel_size = 1`:** Se modifichiamo i kernel a 1×1, la convoluzione perderà completamente la sua proprietà di estrarre "informazione locale" descritta all'inizio del capitolo. Ogni pixel verrà elaborato isolatamente, proprio come accade negli inefficaci modelli MLP. Il modello fallirebbe nel deblurring spaziale, poiché non riuscirebbe a vedere i pixel adiacenti fusi insieme dalla sfocatura.

- **Aumento a `kernel_size = 5` o `7`:** Aumentando la dimensione, la singola operazione convoluzionale guarderà un "intorno" più ampio di pixel per ogni operazione (aumenta il campo ricettivo locale). Questo permetterebbe alla rete di "capire" meglio la struttura della sfocatura 9×9 dell'operatore K, migliorando potenzialmente la ricostruzione e catturando artefatti più ampi. Tuttavia, i kernel più grandi aumentano significativamente i parametri da addestrare per ogni filtro (passando da 9 parametri per filtro a 25 o 49), rendendo la rete più pesante computazionalmente, motivo per cui si preferiscono spesso molti livelli con kernel 3×3 impilati rispetto a pochi livelli con kernel grandi.

Residual Learning, Receptive Field, and UNet

▼ Esercizio di codice: modifica il numero base di canali in una o più varianti e confronta sia il numero di parametri che la qualità della ricostruzione.

Aumentando il numero base di canali (ad esempio passando da 32 a 64 filtri nel primo livello), il numero totale di parametri della rete crescerà in modo quadratico, rendendo il modello computazionalmente molto più pesante (maggiore memoria GPU richiesta e tempi di training dilatati). A livello di qualità, un modello con più canali ha una maggiore "capacità espressiva" e solitamente produce ricostruzioni di qualità superiore e con dettagli più accurati. Tuttavia, se aumentato eccessivamente rispetto ai dati a disposizione, il modello rischia di andare in *overfitting*, memorizzando il rumore di addestramento anziché generalizzare.

▼ Esercizio di codice: ripeti il confronto finale su una diversa immagine di test corrotta e verifica se il comportamento relativo delle varianti cambia.

Testare le reti su immagini di test diverse è l'unico modo per valutarne la vera **generalizzazione**. Spesso, modelli molto complessi (come UNet++ o Attention UNet) possono performare in modo eccellente su un'immagine ma crollare su un'altra se l'anatomia o l'angolo della corruzione sono leggermente diversi. Verificando su un set di test eterogeneo, si nota spesso che l'architettura che vince su una specifica immagine non è sempre la vincitrice assoluta; le prestazioni relative cambiano in base alla robustezza del modello agli out-of-distribution (anomalie non viste nel training).

Vision Transformers and Loss Design

▼ Esercizio di codice: nell'esempio ViT, cambia la dimensione della patch da 16 a 8 o 32. Come cambiano il numero di token e il costo computazionale?

Assumendo un'immagine in input di dimensioni 256×256:

- **Patch = 16:** L'immagine è divisa in una griglia 16×16, generando **256 token**.
- **Patch = 8:** L'immagine viene divisa in una griglia 32×32, generando **1024 token**. Il numero di token quadruplica. Poiché la complessità della self-attention è $O(N^2)$, il costo computazionale non aumenta di 4 volte, ma di **16 volte** (4^2), rendendo il modello molto più pesante e lento.
- **Patch = 32:** La griglia diventa 8×8, generando solo **64 token**. Il costo computazionale precipita a circa $1/16$ di quello originale, ma l'immagine viene scomposta in blocchi così grandi che la rete perderà gran parte della sua capacità di ricostruire i dettagli anatomici o strutturali fini.

▼ Esercizio di codice: sostituisci MSELoss con `torch.nn.L1Loss()` in uno degli esempi di ricostruzione e confronta l'output visivo.

In pratica, sostituendo il criterio nell'ottimizzatore (`loss_fn = torch.nn.L1Loss()`) e ri-addestrando il modello, noteresti un chiaro miglioramento visivo. L'immagine finale non cambierebbe nella geometria generale, ma guardando da vicino i confini (es. contorni degli organi in una CT medica o artefatti ad alta frequenza), l'output della L1Loss apparirebbe molto più croccante e definito. I dettagli che prima venivano "sfumati" o fusi insieme a causa dell'approccio penalizzante quadratico della MSE verrebbero preservati con maggiore nitidezza dalla L1.

Cross-Domain End-to-End Reconstruction

▼ Esercizio di codice: modifica il numero di angoli di proiezione nell'operatore CT e osserva come cambiano il sinogramma e l'immagine FBP.

Se riduci il numero di angoli di proiezione (passando, ad esempio, da una TC completa con centinaia di proiezioni a una vista sparsa con soli 30 o 50 angoli):

- **Sinogramma:** Apparirà più frammentato, mostrando un netto sottocampionamento sull'asse angolare (vuoti o bande nere orizzontali/verticali a seconda dell'orientamento).
- **Immagine FBP:** Subirà un forte degrado. Poiché non ci sono abbastanza proiezioni per annullare correttamente le frequenze indesiderate durante la retroproiezione, l'immagine anatomica verrà oscurata da pesanti artefatti a raggiera (*streaking artifacts*) e rumore, rendendo molto più difficile il successivo lavoro di pulizia della U-Net.

▼ Esercizio di codice: modifica la larghezza della UNet alterando `base_ch` e confronta la qualità della ricostruzione rispetto al costo computazionale.

Il parametro `base_ch` definisce il numero di canali (filtri) nel primo livello della rete, che a sua volta determina la "larghezza" di tutti i livelli successivi.

- **Aumentando `base_ch` (es. da 32 a 64):** Il numero totale di parametri della U-Net cresce in modo quadratico. Il costo computazionale (memoria VRAM richiesta e tempo di addestramento) aumenterà drasticamente. Tuttavia, la rete avrà una capacità espressiva molto maggiore, riuscendo a rimuovere artefatti più complessi e portando tipicamente a una qualità di ricostruzione finale superiore (PSNR più alto).
- **Riducendo `base_ch` (es. a 16):** La rete diventerà estremamente veloce e leggera, ma perderà molta della sua capacità di riconoscere pattern complessi. La qualità della ricostruzione si degraderà, e la rete potrebbe fallire nel correggere le striature più gravi o tendere a sfocare troppo l'immagine.

Deep Generative Models, VAEs, and GANs

▼ Esercizio di codice: cambia la dimensione latente da 128 a 32 o 256 e confronta la qualità dei campioni VAE e GAN.

Il documento sottolinea l'importanza di mantenere una dimensione latente *moderata* affinché il collo di bottiglia non possa semplicemente memorizzare troppe informazioni.

- **Riduzione a 32:** Imporrebbe un collo di bottiglia molto stretto. La rete sarebbe costretta a scartare troppe informazioni e le immagini generate/ricostruite potrebbero risultare eccessivamente sfocate o prive di dettagli fini.
- **Aumento a 256:** Permetterebbe alla rete di ricostruire le immagini in modo eccellente, ma rischierebbe di degradare la qualità del modello generativo (specialmente nel VAE): l'encoder potrebbe ignorare il prior gaussiano e "memorizzare" il dataset, comportandosi come un semplice AutoEncoder. Di conseguenza, il campionamento di nuovi vettori latenti casuali produrrebbe campioni di scarsa qualità.

▼ Esercizio di codice: varia il parametro di regolarizzazione λ nell'esempio del problema inverso latente e osserva come cambia la ricostruzione.

Nell'ottimizzazione del Deep Generative Prior (DGP), la formula include un termine di regolarizzazione $\lambda||z||_2^2$ (o equivalente per il prior latente) che bilancia la fedeltà ai dati corrotti misurati e il realismo indotto dal generatore.

- Se λ è **troppo piccolo**, l'ottimizzazione cercherà in tutti i modi di adattare l'immagine misurata y^δ, forzando lo spazio latente a generare un'immagine che "spieghi" il rumore e gli artefatti, perdendo il beneficio del prior generativo appreso.

- Se λ è **troppo grande**, l'ottimizzazione penalizzerà fortemente qualsiasi deviazione dal centro dello spazio latente (il prior $\mathcal{N}(0, I)$), ignorando parzialmente il termine di fedeltà ai dati. Questo risulterebbe in un'immagine generica, eccessivamente liscia o vicina alla "media" del dataset, ma che manca di corrispondere ai dettagli anatomici specifici del dato misurato.

👨‍💻 Diffusion Models: DDPM and DDIM

▼ Esercizio di codice: cambia il numero di timestep di diffusione da 200 a 100 o 400 e osserva come cambiano addestramento e campionamento.

- **Riducendo a 100 timestep:** Il *noise schedule* diventa più ripido e aggressivo. Questo rende il compito di denoising della rete più difficile per ogni singolo passo, poiché le differenze tra uno stato e l'altro sono più ampie. Tuttavia, il tempo di campionamento inverso puro (senza DDIM) si dimezzerà.
- **Aumentando a 400 timestep:** La transizione tra i livelli di rumore diventa molto più dolce e fine, facilitando l'addestramento della rete nel cogliere le piccole variazioni. Il compromesso è che il campionamento standard impiegherà il doppio del tempo, rendendo il processo più accurato ma computazionalmente più oneroso.

▼ Esercizio di codice: prova il campionamento DDIM con $\eta > 0$ e confronta le immagini generate con il caso deterministico $\eta = 0$.

Il parametro η controlla il grado di stocasticità (casualità) introdotto durante il campionamento inverso.

- Con $\eta = 0$, il processo DDIM è completamente deterministico: se parti dallo stesso rumore latente, arriverai inesorabilmente alla stessa identica immagine generata.
- Con $\eta > 0$ (fino a $\eta = 1$, che recupera la varianza originale del DDPM), viene reintrodotto rumore casuale a ogni step. Questo significa che, partendo dallo stesso rumore latente iniziale, le traiettorie di generazione divergeranno leggermente, producendo variazioni visive e dettagli strutturali differenti ogni volta che si esegue il campionamento.

👨‍💻 Diffusion Models for Inverse Problems

▼ Esercizio di codice: modifica l'angolo di sfocatura o la dimensione del kernel e confronta come reagiscono i due metodi.

Modificare l'angolo o la dimensione del kernel altera il Modello Forward (K). Entrambi i metodi dipendono strettamente dalla precisione di questo operatore fisico. Se il modello di acquisizione impostato diventa molto più severo, la qualità della ricostruzione può deteriorarsi. Un passaggio di denoising o un prior sbilanciato rispetto a un problema inverso reso più complesso potrebbe portare l'IA a generare dettagli plausibili ma errati (allucinazioni) per compensare la perdita di informazione.

▼ Esercizio di codice: varia la scala di guida DPS o il parametro `tau` di DiffPIR e osserva l'effetto sulla qualità della ricostruzione.

- **Per il DPS (Forza della guida):** Se la scala di guida è troppo piccola, la ricostruzione può ignorare le misurazioni e deviare verso immagini visivamente plausibili ma incoerenti con i dati. Se è troppo grande, il campionatore può diventare instabile o produrre risultati eccessivamente nitidi (oversharpened).
- **Per DiffPIR (Parametro `tau`):** Il parametro τ agisce come la dimensione di un passo di ottimizzazione per la coerenza dei dati. Se il passaggio di denoising è troppo dominante (basso τ), il risultato può allucinare dettagli errati. Se il passaggio di coerenza dei dati è troppo forte (alto τ), il metodo può diventare eccessivamente levigato (oversmoothed) o oscillatorio.

▼ Esercizio di codice: genera due diverse ricostruzioni DPS da inizializzazioni casuali diverse e confrontale. Cosa dice questo sul campionamento posteriore?

In teoria, un vero campionatore posteriore dovrebbe rappresentare l'incertezza del problema, fornendo diverse possibili soluzioni valide per lo stesso dato corrotto. Eseguendo il codice con due rumori iniziali diversi, ci si aspetterebbe di ottenere due immagini con lievi variazioni nei dettagli non risolvibili dalle misurazioni. Tuttavia, nella pratica, si verifica spesso il "Collasso dell'Incertezza", dove molte implementazioni semplificate collassano verso una singola soluzione preferita indipendentemente dall'inizializzazione. Inoltre, un'immagine generata in questo modo può apparire molto convincente anche quando non rappresenta un corretto campione posteriore.